

CAMPUS TASKS

Architecture Micro-Logicielle, Sécurité des Systèmes d'Information et Automatisation des Cycles de Livraison (CI/CD)

- **Autrice** : Marième Ndiaye
- **Établissement** : IIBS — Institut Informatique Business School
- **Date** : Octobre 2026
- **Version** : 1.0.0
- **Sous la supervision du corps professoral du Département Génie Logiciel**

TABLE DES MATIÈRES

1. Introduction Générale, Problématique et Cahier des Charges
2. Analyse Fonctionnelle, Spécifications et Modélisation Métier
3. Architecture Logicielle et Patrons de Conception (Design Patterns)
4. Sécurité Avancée, Cryptographie et Isolation des Données (Anti-IDOR)
5. Couche de Persistance Relationnelle et Optimisation des Requêtes SQL
6. Ingénierie Frontend et Cycle de Vie Mobile avec Flutter & Dart
7. Virtualisation par Conteneurs : Processus Docker et Orchestration Compose
8. Automatisation Industrielle : Pipelines d'Intégration et de Déploiement Continus (CI/CD)
9. Protocole de Validation, Plan de Soutenance et Scénarios de Test Audités
10. Conclusion Générale, Limites du Système et Perspectives d'Évolution

1. INTRODUCTION GÉNÉRALE, PROBLÉMATIQUE ET CAHIER DES CHARGES

1.1 Contexte du projet

Dans le parcours académique d'un étudiant en enseignement supérieur, la multiplicité des unités d'enseignement engendre une dispersion critique des flux d'informations. Un étudiant de Licence 3 doit manager simultanément des livrables de natures diverses : projets de programmation, révisions d'examens, devoirs sur table et rapports de recherche. L'absence d'outils centralisés conduit à une désorganisation cognitive et à une augmentation statistique des risques d'échéances manquées.

1.2 Problématique logicielle

Les solutions de gestion de tâches génériques du marché ne répondent pas aux contraintes structurelles du monde universitaire, notamment le regroupement des tâches par matières et le calcul d'indicateurs de retard spécifiques aux coefficients académiques. Le projet **CampusTasks** naît de la nécessité de concevoir une application mobile dédiée, hautement disponible, capable de fonctionner en mode déconnecté et garantissant une étanchéité absolue des données entre les utilisateurs.

1.3 Objectifs techniques et périmètre L3

D'un point de vue académique, ce projet ne se limite pas à la simple production d'une interface graphique. Il impose la validation des compétences d'ingénierie logicielle requises en Licence 3 :

- Séparation stricte des responsabilités par une architecture distribuée (découplage Frontend/Backend).
- Implémentation de mécanismes de sécurité industrielle (Jetons d'accès, chiffrement).
- Optimisation algorithmique des requêtes d'accès aux bases de données relationnelles.
- Industrialisation du cycle de vie du logiciel par la conteneurisation et l'intégration continue.

2. ANALYSE FONCTIONNELLE, SPÉCIFICATIONS ET MODÉLISATION MÉTIER

2.1 Analyse des Besoins Fonctionnels (Matières et Tâches)

L'application doit répondre à un ensemble d'exigences fonctionnelles strictes articulées autour de trois modules principaux :

- **Module d'Authentification** : Permettre la création de comptes utilisateurs uniques et la reconnexion sécurisée.
- **Module de Gestion Académique** : Permettre la création, la modification et la suppression de matières, agissant comme des conteneurs logiques de tâches.
- **Module d'Orchestration des Tâches** : Permettre l'affectation d'attributs précis à chaque tâche (titre, description, date limite de rendu, niveau de priorité, statut de complétion).

2.2 Le Tableau de Bord Évolutif (Dashboard)

Le tableau de bord constitue le cœur décisionnel de l'interface. Il doit calculer en temps réel des agrégats de données à partir de la persistance :

- Le décompte exact des tâches selon leur état (à faire, en cours, terminées).

- L'isolation prioritaire des prochaines échéances basées sur une comparaison temporelle dynamique.
- L'extraction des tâches en retard de complétion par rapport à l'horodatage système actuel du serveur.

2.3 Modélisation Conceptuelle des Données (MCD)

Le modèle relationnel repose sur trois entités majeures connectées par des contraintes d'intégrité référentielle fortes :

- **Entité User (Utilisateur)** : Définit l'identité du client (identifiant unique, pseudonyme, empreinte du mot de passe). Elle est à la racine de la structure.
- **Entité Subject (Matière)** : Représente une unité d'enseignement. Elle est liée à l'utilisateur par une relation de un-à-plusieurs (1:N). Une matière ne peut appartenir qu'à un seul utilisateur.
- **Entité Task (Tâche)** : Représente le livrable. Elle possède une relation de plusieurs-à-un (N:1) avec l'entité Subject. De plus, pour renforcer la sécurité au niveau atomique, elle maintient une relation directe de plusieurs-à-un (N:1) avec l'entité User (le propriétaire ou owner).

3. ARCHITECTURE LOGICIELLE ET PATRONS DE CONCEPTION (DESIGN PATTERNS)

3.1 Architecture Découplée et Flux REST

Le système implémente le modèle architectural **Client-Serveur distribué**. Le frontend et le backend sont totalement indépendants, communiquant exclusivement via le protocole applicatif HTTP/HTTPS à travers une interface de programmation (API) de type REST. Les échanges de données sont standardisés sous la forme de documents légers JSON (JavaScript Object Notation). Cette configuration permet de faire évoluer le client mobile ou le serveur sans impacter le composant adjacent.

3.2 L'Architecture en Couches du Backend

Au sein du framework Spring Boot, le code est segmenté selon un modèle d'architecture en couches à flux unidirectionnel :

Couche Contrôleur (Web) → Couche Service (Métier) → Couche Repository (Persistance) → Base de Données

- **La Couche Contrôleur (Contrôleurs REST)** : Elle intercepte les requêtes HTTP entrantes. Sa responsabilité unique est de valider la structure des données reçues (via des mécanismes de Bean Validation), de mapper les URLs versionnées (sous le préfixe

réglementaire /api/v1), et de déléguer le traitement à la couche inférieure. Elle retourne systématiquement des réponses standardisées basées sur les codes de statut HTTP (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found).

- **La Couche Service (Services Métier)** : Elle centralise l'intelligence applicative. C'est ici que s'exécutent les algorithmes de validation métier, la gestion transactionnelle (validation ou annulation des écritures) et les vérifications de droits d'accès. Elle isole totalement la logique logicielle des protocoles de transport (HTTP) ou de persistance (SQL).
- **La Couche Repository (Accès aux données)** : Elle est composée d'interfaces s'appuyant sur Spring Data JPA. Elle abstrait les requêtes SQL complexes en manipulant des objets du domaine via un mapping objet-relationnel (ORM).

3.3 Utilisation des Data Transfer Objects (DTO)

Afin d'éviter l'exposition directe des entités de la base de données sur le réseau (ce qui constituerait une faille de sécurité majeure par divulgation d'informations), le système met en place le patron de conception **DTO (Data Transfer Object)**. Des objets spécifiques sont créés pour la réception des formulaires (ex: requêtes d'inscription) et pour l'envoi des données filtrées vers le mobile. Cette couche d'abstraction permet de modifier la structure des tables SQL sans jamais impacter le contrat d'interface signé avec l'application Flutter.

4. SÉCURITÉ AVANCÉE, CRYPTOGRAPHIE ET ISOLATION DES DONNÉES (ANTI-IDOR)

4.1 Authentification Stateless par Jetons JWT

Dans une architecture distribuée moderne, la conservation des sessions utilisateurs en mémoire serveur (Stateful) limite drastiquement la montée en charge. Le projet implémente donc une authentification **Stateless (sans état)** basée sur le standard **JWT (JSON Web Token)**.

Le processus suit un cycle rigoureux : lors de la soumission des identifiants par l'application mobile, le serveur vérifie la validité des informations. Si elles sont correctes, il forge un jeton composé de trois parties distinctes séparées par des points : l'en-tête (indiquant l'algorithme de chiffrement), le corps ou charge utile (contenant les revendications de l'utilisateur comme son pseudo, sa date d'émission et sa date d'expiration) et la signature.

La signature est générée en chiffrant l'en-tête et le corps à l'aide d'un algorithme de hachage cryptographique **HMAC (SHA-256)** combiné à une clé secrète de haute sécurité d'au moins 32 caractères stockée uniquement sur le serveur. Chaque requête HTTP ultérieure envoyée par le mobile devra inclure ce jeton dans son en-tête d'autorisation sous le type Bearer.

4.2 Cryptographie et Hachage des Mots de Passe (BCrypt)

Le stockage de mots de passe en clair ou via des algorithmes de hachage simples (comme MD5 ou SHA-1, aujourd'hui vulnérables aux attaques par dictionnaire ou tables de correspondance) est formellement proscrit. Le système CampusTasks intègre la fonction de dérivation de clé **BCrypt**.

BCrypt intègre nativement un mécanisme de "sel" (salt), une chaîne de caractères aléatoires ajoutée au mot de passe avant le hachage, rendant impossible l'utilisation de tables de décodage précalculées. De plus, il implémente un facteur de coût adaptatif (work factor) qui ralentit volontairement le calcul de l'empreinte. Cela protège efficacement le système contre les attaques par force brute ou par calcul distribué, car la validation d'une signature exige un temps de calcul prohibitif pour un attaquant extérieur.

4.3 Protection Impénétrable contre les Failles de Type IDOR

La faille **IDOR (Insecure Direct Object Reference)** se produit lorsqu'un système informatique permet à un utilisateur connecté d'accéder à une ressource en fournissant simplement son identifiant numérique (ex: GET /api/v1/tasks/15), sans vérifier si cette ressource lui appartient. Un attaquant pourrait incrémenter l'ID dans une boucle pour siphonner l'intégralité de la base de données.

Pour éradiquer ce risque, CampusTasks déploie un protocole d'isolation à double verrou :

1. **Extraction de l'identité sécurisée** : Le framework inspecte le jeton JWT à chaque requête, en extrait de manière sûre l'identité de l'utilisateur connecté, et injecte cette identité dans le contexte de sécurité de l'application. Cette information provient de la signature cryptographique et ne peut pas être falsifiée par l'utilisateur.
2. **Filtrage SQL forcé** : Aucune requête SQL de sélection, de modification ou de suppression de tâche ne repose uniquement sur l'ID de la tâche. La couche de persistance force l'association systématique de l'identifiant de la tâche ET de l'identifiant du propriétaire extrait du jeton. Si un étudiant tente d'accéder à la tâche d'un autre, la base de données retourne un résultat vide, induisant une réponse d'erreur et bloquant immédiatement la tentative d'intrusion.

5. COUCHE DE PERSISTANCE RELATIONNELLE ET OPTIMISATION DES REQUÊTES SQL

5.1 Architecture des requêtes SQL et Filtrage Multicritère Dynamique

La gestion des tâches universitaires requiert une grande flexibilité d'affichage. L'étudiant doit pouvoir trier et filtrer ses échéances selon plusieurs axes (par exemple, afficher uniquement

les tâches d'une matière spécifique, ou uniquement les tâches en cours, ou l'intégralité de ses tâches). La couche de persistance implémente un algorithme de filtrage multicritère.

Le mécanisme évalue la valeur des paramètres de filtrage transmis par le client. Si le filtre de matière ou de statut est évalué à une valeur nulle, le système ignore dynamiquement cette condition dans la clause de restriction de la base de données. En revanche, si une valeur spécifique est soumise, le filtre est immédiatement appliqué. Toutes les requêtes sont triées de manière native par ordre croissant des dates limites de rendu afin de garantir la pertinence chronologique des listes de tâches présentées à l'utilisateur.

5.2 Résolution Optimisée du Problème des Requêtes N+1

Dans les frameworks utilisant des architectures ORM (comme Hibernate/JPA), le problème des requêtes N+1 est l'une des causes principales d'effondrement des performances des applications. Par défaut, lorsqu'une entité (comme Tâche) possède une relation avec une autre entité (comme Matière), l'ORM exécute une première requête SQL pour récupérer la liste des N tâches d'un utilisateur. Ensuite, pour chaque tâche individuelle de cette liste, l'ORM va générer une requête SQL supplémentaire pour charger les détails de la matière associée. Pour une liste de 50 tâches, le système effectuera donc $1 + 50 = 51$ requêtes vers la base de données, saturant le pool de connexions et augmentant drastiquement le temps de réponse.

CampusTasks résout ce problème de manière industrielle en utilisant le mécanisme de **jointure par capture optimisée (Join Fetch)**. Lors de l'appel à la base de données, le moteur JPA génère une requête SQL complexe unique qui intègre une jointure interne entre la table des tâches et la table des matières. La base de données retourne un jeu de résultats global contenant l'intégralité des informations imbriquées. L'ORM peut alors instancier les objets Java en une seule passe. Le nombre de requêtes est ainsi réduit à une seule (1), garantissant des performances optimales et un temps d'accès mémoire minimal, quelle que soit la taille du volume de données traité.

6. INGÉNIERIE FRONTEND ET CYCLE DE VIE MOBILE AVEC FLUTTER & DART

6.1 Choix Technologiques du Framework Multiplateforme

Le choix du framework **Flutter**, propulsé par le langage orienté objet **Dart**, répond à des exigences d'efficacité de développement et de fidélité graphique. Flutter n'utilise pas les composants natifs du système d'exploitation mobile, mais redessine l'intégralité de son interface pixel par pixel à l'aide d'un moteur de rendu haute performance. Cela garantit une cohérence visuelle absolue entre les plateformes Android et iOS. Le langage Dart apporte quant à lui une compilation binaire native (Ahead-Of-Time), éliminant le besoin d'un pont JavaScript intermédiaire et offrant des animations fluides à 60 images par seconde.

6.2 Architecture logicielle du Client Mobile

Pour éviter l'écueil classique des projets juniors consistant à intégrer la logique réseau et les règles de gestion directement au sein des vues de l'interface (Widgets), l'application applique un découpage architectural strict :

- **La Couche Présentation (Widgets)** : Elle est uniquement responsable de l'affichage graphique et de la capture des événements utilisateurs. Elle est totalement asynchrone et réagit aux changements d'états de manière dynamique.
- **La Couche Repository Frontend** : Elle encapsule les appels réseau. Elle utilise un client HTTP configuré pour injecter le jeton JWT stocké de façon sécurisée dans les en-têtes de chaque requête.
- **La Couche Modèles** : Elle convertit les chaînes de caractères brutes JSON reçues du serveur en objets Dart fortement typés, sécurisant la manipulation des données au sein de l'application.

6.3 Processus d'Injection Dynamique de Configuration à la Compilation

L'un des défis majeurs lors du cycle de développement d'une application mobile connectée est la variabilité des adresses réseau du backend. En phase de développement local, l'API est hébergée sur une machine locale, tandis qu'en production, elle est accessible via une URL sécurisée avec un nom de domaine officiel. Modifier manuellement l'adresse IP dans le code source avant chaque compilation est une source majeure d'erreurs et de failles de sécurité.

Le projet résout cette contrainte en implémentant le mécanisme d'**injection de variables d'environnement à la compilation (--dart-define)**. Lors du processus de compilation, l'adresse de base de l'API est passée en argument au compilateur Flutter. Le code source Dart extrait cette valeur depuis son environnement d'exécution d'une manière totalement transparente. Si aucun argument n'est fourni, le système bascule par sécurité sur une valeur par défaut pointant sur la boucle locale.

Ce processus permet de générer des versions de développement ou des versions de production hautement sécurisées (en forçant l'utilisation exclusive du protocole HTTPS) à partir de la même base de code rigoureusement inchangée.

7. VIRTUALISATION PAR CONTENEURS : PROCESSUS DOCKER ET ORCHESTRATION COMPOSE

7.1 Concept de Conteneurisation et Isolation Système

La conteneurisation transforme la méthode de déploiement classique en encapsulant l'application et l'intégralité de ses dépendances systèmes au sein d'une unité logique isolée

appelée **conteneur**. Contrairement à une machine virtuelle traditionnelle qui nécessite l'émulation d'un système d'exploitation complet au-dessus d'un hyperviseur (ce qui consomme énormément de mémoire vive et d'espace disque), les conteneurs partagent le même noyau Linux que la machine hôte. Ils s'isolent au niveau de l'espace utilisateur à l'aide des fonctionnalités du noyau comme les namespaces (pour isoler les processus, le réseau et les fichiers) et les cgroups (pour limiter l'allocation des ressources matérielles comme le processeur et la RAM). Cela garantit un démarrage quasi instantané et une empreinte mémoire extrêmement réduite.

7.2 Implémentation Avancée du Dockerfile Multi-Stage

Pour garantir un niveau de sécurité et de performance de classe industrielle sur le serveur de production, le backend Spring Boot est encapsulé à l'aide d'un processus de **construction multi-stage (Multi-Stage Build)**. Un Dockerfile classique intègre tous les outils nécessaires à la compilation (comme le SDK Java et Maven) au sein de l'image finale, ce qui alourdit considérablement son poids (dépassant souvent le gigaoctet) et introduit des failles de sécurité majeures (la présence de compilateurs simplifie la tâche d'un attaquant ayant réussi à s'introduire dans le conteneur).

Le processus multi-stage résout cette vulnérabilité en segmentant la construction en deux étapes étanches :

1. **L'Étape de Build** : Le conteneur initial est basé sur une image lourde contenant l'environnement Maven et le JDK Java complet. Le code source y est copié, les dépendances sont téléchargées et l'application est compilée pour générer l'archive finale exécutable. Une fois cette étape terminée, ce conteneur est totalement détruit.
2. **L'Étape d'Exécution** : Un second conteneur est initialisé à partir d'une image de production ultra-légère basée sur une distribution Linux minimaliste contenant uniquement un JRE (Java Runtime Environment) épuré de tout outil de développement. Le fichier JAR généré à l'étape précédente y est copié.

L'image finale ainsi produite est extrêmement légère (réduite à quelques mégaoctets) et possède une surface d'attaque logicielle quasi nulle. De plus, par mesure de sécurité, le conteneur applique une politique de moindres privilèges en interdisant l'exécution du processus Java par l'utilisateur racine (root), déléguant l'exécution à un profil système aux droits limités.

7.3 Orchestration Multi-Services avec Docker Compose

L'architecture de l'application CampusTasks nécessite la coordination de deux services distincts : le serveur d'API REST et le serveur de base de données PostgreSQL. L'outil **Docker Compose** assure le rôle d'orchestrateur local en traduisant cette infrastructure au sein d'un unique fichier de configuration descriptif.

Le processus applique des concepts d'ingénierie réseau et système avancés :

- **Réseau Privé Virtuel** : Docker Compose initialise un réseau local isolé et chiffré. Les conteneurs communiquent entre eux à l'aide de résolutions DNS internes basées sur le nom de leurs services. Le conteneur PostgreSQL est totalement retiré du réseau public extérieur : il n'expose aucun port sur la machine hôte, ce qui le protège contre toute tentative de balayage de ports ou d'attaques directes depuis Internet. Seul le conteneur backend possède une double interface lui permettant de recevoir les requêtes HTTP publiques et de communiquer en interne avec la base de données.
- **Persistance par Volumes Isolés** : Par nature, un conteneur est éphémère (si le processus s'arrête ou s'il est mis à jour, toutes les données écrites dans son système de fichiers sont définitivement perdues). Pour garantir la pérennité des données académiques des étudiants, Docker Compose configure un volume persistant indépendant. Ce volume mappe un dossier sécurisé de la machine hôte directement dans le système de stockage interne de PostgreSQL, assurant la survie intégrale des bases de données lors des cycles de mise à jour de l'infrastructure.

7.4 Mécanisme de Synchronisation par Test de Santé (Healthcheck)

Dans une infrastructure conteneurisée standard, le démarrage simultané de tous les services engendre fréquemment des plantages applicatifs appelés "conditions de course" (race conditions). Un conteneur de base de données comme PostgreSQL met plusieurs secondes à s'initialiser lors de son premier lancement (création des fichiers systèmes, vérification des tables, ouverture des ports réseau). Si le conteneur backend Spring Boot démarre en même temps, il tente immédiatement d'établir sa connexion SQL. Ne trouvant pas de base active, l'application Java plante instantanément et s'arrête.

CampusTasks résout ce problème en implémentant un mécanisme de validation d'état de santé dynamique (**healthcheck**). Le conteneur PostgreSQL exécute en interne et à intervalles réguliers une commande système native vérifiant sa capacité réelle à accepter des transactions SQL. Le conteneur backend est configuré pour observer cet indicateur de santé. Son processus de lancement reste en attente tant que le signal d'état n'a pas basculé sur une valeur saine. Une fois le voyant au vert, le conteneur backend démarre sa compilation et sa connexion en toute sécurité, garantissant une tolérance aux pannes totale lors des démarrages à froid de l'infrastructure.

8. AUTOMATISATION INDUSTRIELLE : PIPELINES D'INTÉGRATION ET DE DÉPLOIEMENT CONTINUS (CI/CD)

8.1 Philosophie DevOps et Automatisation des Cycles de Vie

La philosophie DevOps vise à unifier le développement logiciel (Dev) et l'exploitation des infrastructures (Ops) en éliminant les interventions manuelles lors des phases de tests et de déploiement. L'automatisation complète de cette chaîne supprime les erreurs humaines,

standardise la qualité du code et réduit drastiquement le délai de livraison des correctifs en production. Le projet CampusTasks concrétise cette approche en configurant la plateforme **GitHub Actions** comme moteur d'orchestration CI/CD.

8.2 Le Pipeline d'Intégration Continue (Pull Request CI)

Le premier flux de travail est conçu comme une barrière de contrôle qualité automatisée. Il se déclenche de façon obligatoire à chaque fois qu'un développeur soumet une demande de fusion (Pull Request) vers la branche de production principale.

Le pipeline déroule un protocole de tests et d'analyses en trois phases distinctes :

1. **Initialisation de l'environnement** : Le système instancie un serveur virtuel vierge fonctionnant sous la dernière version stable d'Ubuntu Linux. Il y configure le kit de développement Java (JDK) requis ainsi que les caches d'outils pour accélérer les futurs builds.
2. **Validation unitaire du Backend** : L'agent GitHub Actions télécharge le code source et exécute de manière automatisée la suite complète des tests unitaires et d'intégration via Maven. Si un seul test échoue ou si une régression logicielle est détectée, le processus est immédiatement interrompu et la fusion du code est bloquée sur GitHub.
3. **Analyse statique du Frontend** : En parallèle, le pipeline initialise un environnement Flutter et télécharge les dépendances de l'application mobile. Il exécute les outils d'analyse statique du code (linter). Ce processus vérifie le respect des normes de programmation Dart, la propreté de la syntaxe et l'absence de structures obsolètes, garantissant ainsi la maintenabilité future de l'application mobile.

8.3 Le Pipeline de Déploiement Continu (Release Docker)

Le second flux de travail prend en charge la phase de livraison industrielle (Continuous Deployment). Il s'active automatiquement lors d'un push direct sur la branche principale ou dès qu'un administrateur publie un tag de version officiel respectant la nomenclature du versioning sémantique.

Le processus se déroule selon quatre étapes clés :

1. **Authentification Sécurisée par Secrets GitHub** : L'agent CI/CD doit se connecter au registre distant Docker Hub pour y publier l'image de l'application. Pour interdire le stockage d'identifiants en clair au sein du code source (ce qui exposerait le système à des vols de comptes), le pipeline exploite le coffre-fort de sécurité intégré de GitHub (GitHub Secrets). Le pseudonyme et le jeton d'accès personnel de l'utilisateur y sont stockés de manière chiffrée et injectés dynamiquement en mémoire volatile uniquement lors de la phase d'authentification réseau, garantissant une sécurité absolue des accès.
2. **Gestion Dynamique du Versioning et des Métadonnées** : Le pipeline intègre un module d'extraction automatique des métadonnées Git. Ce composant analyse

dynamiquement la nature de l'événement déclencheur. Si le code provient d'un push standard sur la branche principale, il génère une étiquette Docker permettant de mettre à jour la version de référence actuelle. Si le build est déclenché par un tag Git, le module applique un filtre d'expression régulière pour nettoyer la chaîne de caractères (par exemple, transformer automatiquement le tag de version Git en étiquette Docker propre), assurant un versioning sémantique impeccable de l'historique sur le registre.

3. **Construction et Publication (Push) Industrielle** : L'agent compile l'image Docker optimisée en exécutant le Dockerfile multi-stage. Une fois l'image validée localement, elle est poussée vers les serveurs de Docker Hub sous le compte configuré. L'image devient alors instantanément disponible dans le monde entier, prête à être récupérée par n'importe quel serveur de production pour une mise à jour transparente de l'application.

9. PROTOCOLE DE VALIDATION, PLAN DE SOUTENANCE ET SCÉNARIOS DE TEST AUDITÉS

9.1 Scénario d'Établissement du Cycle Utilisateur Complet

Pour démontrer la viabilité du projet devant le jury de Licence 3, la soutenance doit s'articuler autour d'une démonstration pratique et rigoureuse modélisant l'activité de deux étudiants distincts sur le système informatique. La première phase valide le bon fonctionnement du protocole d'authentification et de chiffrement.

Elle déroule l'inscription d'un premier profil sur l'application mobile, sa connexion au système, la vérification de la génération du jeton de sécurité par le serveur, et l'analyse de son stockage sécurisé au sein de la mémoire volatile de l'appareil mobile.

9.2 Validation du Cloisonnement Fonctionnel et du Tableau de Bord

La seconde phase de la démonstration technique prouve la pertinence métier du système et la validité des algorithmes d'affichage. L'utilisateur connecté enregistre plusieurs matières universitaires au sein de son profil, puis y affecte une série de tâches d'échéances et de statuts divers : des tâches terminées, des tâches en cours d'exécution et des tâches volontairement configurées avec des dates limites antérieures à l'heure actuelle du système afin de simuler des retards.

Le plan de validation exige alors de vérifier la mise à jour instantanée des indicateurs du tableau de bord de l'application mobile. Le système doit afficher les calculs exacts des compteurs d'états et isoler correctement les alertes de retards, validant l'exactitude des algorithmes métiers implémentés.

9.3 Simulation d'une Attaque Cyber et Audit de l'Isolation du Système

L'étape la plus critique de la validation consiste à apporter la preuve absolue de l'étanchéité du système face aux attaques de type élévation de privilèges ou falsification de requêtes (IDOR).

Le protocole d'audit se déroule selon le schéma suivant :

1. Un second compte utilisateur distinct est initialisé sur un autre terminal ou via un client de test réseau. Ce compte crée son propre espace de travail, qui doit initialement apparaître totalement vierge de toute tâche ou matière.
2. L'opérateur capture l'identifiant numérique unique d'une tâche appartenant exclusive au premier utilisateur.
3. En utilisant les outils de test réseau, le second utilisateur forge une requête HTTP de modification ou de lecture en ciblant directement cet identifiant numérique, mais en transmettant dans les en-têtes son propre jeton d'authentification valide.
4. L'API REST interceptant la requête doit appliquer ses mécanismes de sécurité : elle extrait l'identité du second utilisateur depuis le jeton, constate que le couple identifiant de ressource/identifiant de propriétaire est invalide en base de données, et rejette immédiatement la requête en retournant un code d'erreur d'interdiction d'accès. La tentative d'attaque est ainsi neutralisée, apportant la preuve scientifique de la sécurité du système.